



ASCII

Sorting



PENGERTIAN SORTING

Sorting merupakan Pengurutan data dalam struktur data sangat penting terutama untuk data yang bertipe data numerik ataupun karakter. Pengurutan dapat dilakukan secara ascending (dari yang terkecil ke terbesar) dan descending (dari yang terbesar ke terkecil). Pengurutan (Sorting) adalah proses pengurutan data yang sebelumnya disusun secara acak sehingga tersusun secara teratur menurut aturan tertentu. Sama seperti halnya metode metode lain yang ada pada pemrograman, sorting juga tidak lepas kaitannya dengan kompleksitas waktu atau efisiensi program, oleh karena itu penting untuk memilih metode mana yang terbaik. Dalam pengurutan data terdapat berbagai macam metode, beberapa diantaranya yaitu

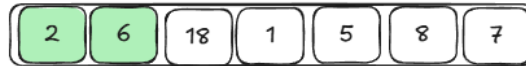
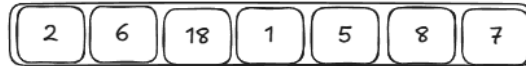
1. Bubble Sort

Bubble sort merupakan algoritma pengurutan yang memiliki konsep pertukaran elemen, bubble akan melihat elemen awal dengan elemen setelahnya lalu melakukan pertukaran apabila elemen setelah lebih besar/lebih kecil bergantung dengan tipe pengurutan ascending/decending



bubble sort

To move canvas, hold o



sudah urut, tidak ditukar



sudah urut, tidak ditukar



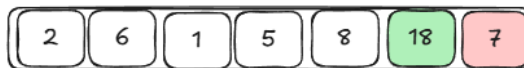
tidak urut, ditukar



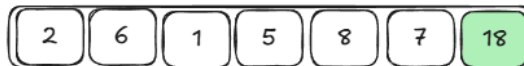
tidak urut, ditukar



tidak urut, ditukar



tidak urut, ditukar



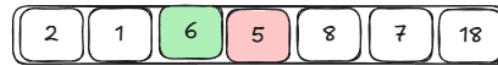
ulang



sudah urut, tidak ditukar



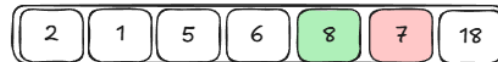
tidak urut, ditukar



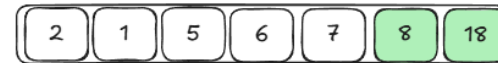
tidak urut, ditukar



sudah urut, tidak ditukar



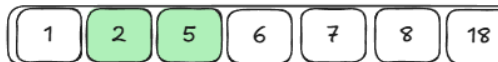
tidak urut, ditukar



sudah urut, ulang dari awal



tidak urut, ditukar



sudah urut, tidak ditukar



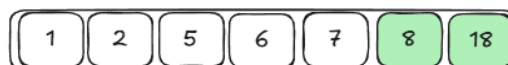
sudah urut, tidak ditukar



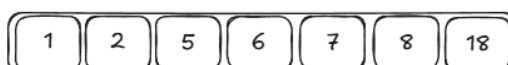
sudah urut, tidak ditukar



sudah urut, tidak ditukar



sudah urut, tidak ditukar



hasil akhir



Adapun untuk implementasinya pada kode di bawah ini:

```
#include <iostream>
using namespace std;

// Inisialisasi Array Global
int arr[] = {64, 34, 25, 12, 22, 11, 90};
int n = sizeof(arr) / sizeof(arr[0]);

void bubbleSort(int arr[], int n) {
    bool swapped;
    // Loop luar untuk jumlah lintasan (pass)
    for (int i = 0; i < n - 1; i++) {
        swapped = false;

        // Loop dalam untuk membandingkan elemen yang bersebelahan
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Tukar elemen jika elemen kiri lebih besar dari
                kanan
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
    }

    // Optimasi: Jika tidak ada pertukaran, berarti array sudah
    urut
    if (swapped == false)
        break;
}

void tampilkanArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
}
```



```
    }  
    cout << endl;  
}  
  
int main() {  
    cout << "Data Sebelum Bubble Sort: ";  
    tampilkanArray(arr, n);  
  
    bubbleSort(arr, n);  
  
    cout << "Data Setelah Bubble Sort : ";  
    tampilkanArray(arr, n);  
  
    return 0;  
}
```

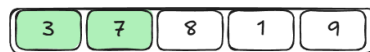
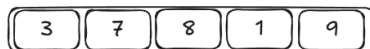
Algoritma ini akan terus berulang sampai seluruh elemen memenuhi syarat, dengan adanya konsep perulangan ini, bubble sort merupakan salah satu algoritma dengan kompleksitas waktu yang paling lambat karena kompleksitasnya adalah $O(n^2)$, berikut ilustrasi bubble sort:



2. Selection Sort

Selection sort merupakan cara pengurutan dengan mencari elemen terkecil terlebih dahulu yang menggunakan elemen pertama sebagai pembanding untuk mencari elemen terkecil, setelah menemukan elemen terkecil maka akan melakukan increment jika syarat terpenuhi baik ascending maupun descending

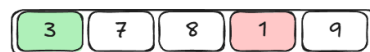
selection sort



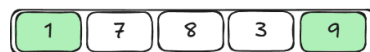
bandingkan, 3 masih yang terkecil



bandingkan, 3 masih yang terkecil



bandingkan, 1 menjadi yang terkecil, terjadi pertukaran posisi



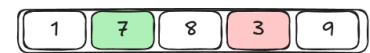
bandingkan, 1 masih yang terkecil



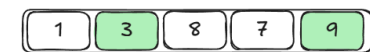
ulangi dengan dimulai dari indeks 1



bandingkan, 7 masih yang lebih kecil



bandingkan, 3 menjadi yang lebih kecil, terjadi pertukaran posisi



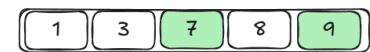
bandingkan, 3 masih yang lebih kecil



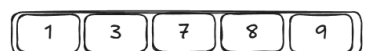
ulangi, dimulai dari indeks 2



bandingkan, 7 menjadi yang lebih kecil, terjadi pertukaran posisi



bandingkan, 7 masih yang lebih kecil



selesai



```
#include <iostream>
using namespace std;

// Inisialisasi Array Global
int arr[] = {29, 10, 14, 37, 13};
int n = sizeof(arr) / sizeof(arr[0]);

void selectionSort(int arr[], int n) {
    // Loop luar untuk memindahkan batas sub-array yang belumurut
    for (int i = 0; i < n - 1; i++) {
        // Anggap elemen pertama dari bagian belumurut sebagai yang
        // terkecil
        int indeksMin = i;

        // Loop dalam untuk mencari elemen terkecil di sisa array
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[indeksMin]) {
                indeksMin = j; // Simpan indeks elemen yang lebih kecil
            }
        }

        // Tukar elemen terkecil yang ditemukan dengan elemen di posisi i
        if (indeksMin != i) {
            swap(arr[i], arr[indeksMin]);
        }
    }
}

void tampilkanArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    cout << "Sebelum Selection Sort: ";
    tampilkanArray(arr, n);
}
```



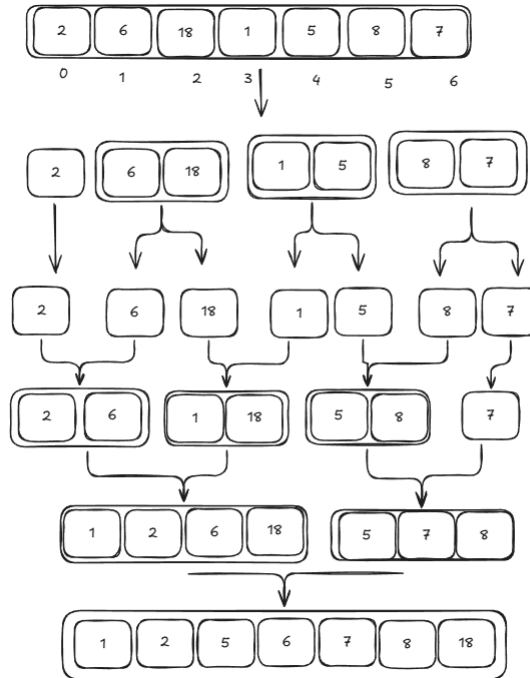

```
selectionSort(arr, n);  
  
cout << "Sesudah Selection Sort : ";  
tampilkanArray(arr, n);  
  
return 0;  
}
```

Selection sort memiliki kompleksitas yang konstan untuk setiap kasusnya baik best case maupun worst case dengan kompleksitas $O(n^2)$



3. Merge Sort

Merupakan salah satu teknik sorting yang menggunakan algoritma Divide and Conquer yang dimana cara kerja dari teknik yaitu memecah kumpulan array sehingga semua terpisah yang selanjutnya akan diurutkan dan digabungkan kembali. Berikut adalah ilustrasi dari merge sort:



```
#include <iostream>
using namespace std;

// Array Global agar mudah diakses oleh semua fungsi
int arr[] = {38, 27, 43, 3, 9, 82, 10, 55};
int n = sizeof(arr) / sizeof(arr[0]);

// Fungsi untuk menggabungkan dua sub-array yang sudah terbagi
void merge(int arr[], int l, int m, int r) {
    int temp[8]; // Array pembantu sementara
    int i = l;   // Indeks awal sub-array kiri
```



```
int j = m + 1; // Indeks awal sub-array kanan
int k = 0;      // Indeks awal array temp

// Bandingkan elemen kiri dan kanan, masukkan yang terkecil ke temp
while (i <= m && j <= r) {
    if (arr[i] < arr[j]) {
        temp[k] = arr[i];
        i++;
    } else {
        temp[k] = arr[j];
        j++;
    }
    k++;
}

// Jika sub-array kiri masih ada sisa, masukkan ke temp
while (i <= m) {
    temp[k] = arr[i];
    i++;
    k++;
}

// Jika sub-array kanan masih ada sisa, masukkan ke temp
while (j <= r) {
    temp[k] = arr[j];
    j++;
    k++;
}

// Salin kembali data dari temp ke array asli (arr)
for (int x = 0; x < k; x++) {
    arr[l + x] = temp[x];
}
}

// Fungsi rekursif untuk membagi array (Divide)
void mergeSort(int arr[], int l, int r) {
```



```
if (l < r) {  
    // Cari titik tengah  
    int m = (l + r) / 2;  
  
    // Bagi terus sisi kiri dan kanan  
    mergeSort(arr, l, m);  
    mergeSort(arr, m + 1, r);  
  
    // Gabungkan kembali (Conquer & Combine)  
    merge(arr, l, m, r);  
}  
}  
  
int main() {  
    cout << "Sebelum sorting: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
  
    // Panggil fungsi merge sort  
    mergeSort(arr, 0, n - 1);  
  
    cout << "Sesudah sorting: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
  
    return 0;}
```

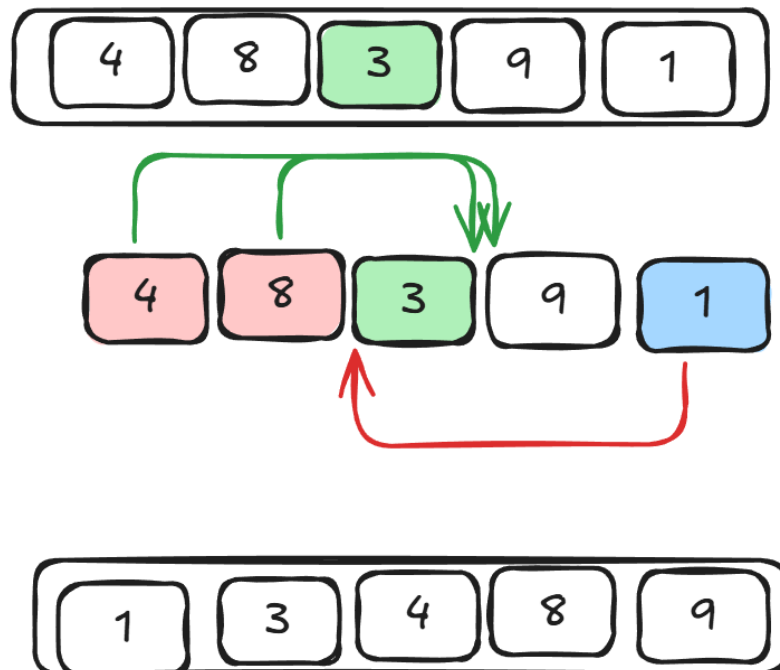
Merge sort selalu memiliki kompleksitas $O(n \log n)$ dikarenakan sekalipun data yang paling acak diberikan kepada metode/algorithm ini merge sort akan tetap membagi 2 data sampai akhirnya menjadi data tunggal yang akan kembali digabungkan berdasarkan tipe penyortiran (ascending/descending).



4. Quick Sort

Sama seperti Merge Sort dimana teknik ini juga menggunakan algoritma divide and conquer dimana cara kerja dari teknik yaitu dengan mengambil salah satu elemen yang akan disebut sebagai pivot dan mempartisi (membagi) array yang berada di sekitar pivot, lalu akan dilakukan pengurutan berdasarkan perbandingan elemen yang lebih kecil atau lebih besar dari pivot. Berikut adalah ilustrasi dari quick sort:

QUICK SORT





```
#include <iostream>
using namespace std;

// Inisialisasi Array Global
int arr[] = {3, 4, 2, 9, 8, 6, 5, 7};
int n = sizeof(arr) / sizeof(arr[0]);

void quickSort(int arr[], int low, int high) {
    // Base case: jika pointer low melewati high, berhenti
    if (low >= high) return;

    // Menentukan data tengah sebagai pivot (Strategi yang lebih stabil)
    int mid = low + (high - low) / 2;
    int pivot = arr[mid];
    int i = low, j = high;

    // Proses Partisi
    while (i <= j) {
        // Cari elemen di kiri yang seharusnya di kanan pivot
        while (arr[i] < pivot) {
            i++;
        }
        // Cari elemen di kanan yang seharusnya di kiri pivot
        while (arr[j] > pivot) {
            j--;
        }

        // Tukar elemen jika ditemukan posisi yang salah
        if (i <= j) {
            swap(arr[i], arr[j]);
            i++;
            j--;
        }
    }

    // Rekursi untuk sub-array bagian kiri (dari low sampai j)
    if (low < j) {

```



```
        quickSort(arr, low, j);
    }
    // Rekursi untuk sub-array bagian kanan (dari i sampai high)
    if (i < high) {
        quickSort(arr, i, high);
    }
}

int main() {
    cout << "Data Belum Terurut: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    // Memanggil fungsi Quick Sort
    quickSort(arr, 0, n - 1);

    cout << "Hasil Quick Sort : ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

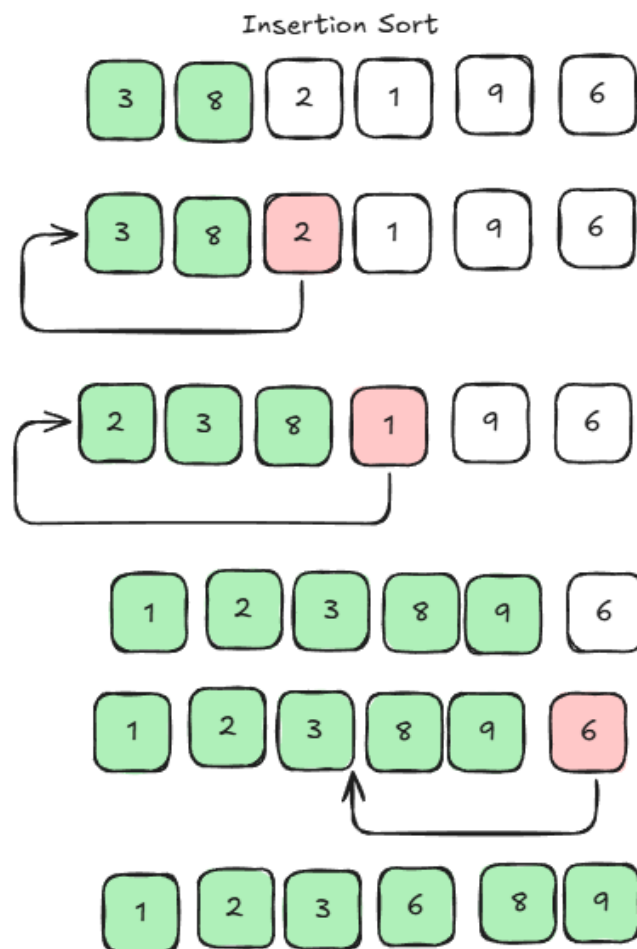
    return 0;
}
```

Jika membahas mengenai quick sort tentunya kita akan mengetahui bahwa quicksort merupakan salah satu algoritma tercepat apabila pivot yang digunakan merupakan nilai tengah data, tetapi apabila data sudah terurut dan memilih pivot yang paling ujung, hal ini akan memberikan worst case yaitu kompleksitas waktu $O(n^2)$



5. Insertion Sort

Insertion Sort merupakan sebuah teknik pengurutan dengan cara membandingkan dan mengurutkan dua data pertama pada array, kemudian membandingkan data para array berikutnya apakah sudah berada di tempat semestinya. Algorithma insertion sort seperti proses pengurutan kartu yang berada di tangan kita. Algorithma ini dapat mengurutkan data dari besar ke kecil (Ascending) dan kecil ke besar (Descending).





```
#include <iostream>
#include <string>

using namespace std;

// Struktur Data Buku
struct Buku {
    int idBuku;
    string judul;
};

// Fungsi Insertion Sort
void insertionSort(Buku rak[], int n) {
    // Loop luar: mengambil buku satu per satu mulai dari buku kedua
    for (int i = 1; i < n; i++) {
        Buku key = rak[i];
        int j = i - 1;

        /* Loop dalam: Membandingkan 'key' dengan buku di kirinya.
           Jika buku di kiri lebih besar, geser ke kanan.
        */
        while (j >= 0 && rak[j].idBuku > key.idBuku) {
            rak[j + 1] = rak[j];
            j = j - 1;
        }
        // Sisipkan buku 'key' di posisi yang benar
        rak[j + 1] = key;
    }
}

void tampilkanRak(Buku rak[], int n) {
    for (int i = 0; i < n; i++) {
        cout << rak[i].idBuku << " | " << rak[i].judul << endl;
    }
    cout << endl;
}
```



```
int main() {
    int n = 5;

    // SCENARIO 1: BEST CASE (O(n))
    // Data sudah urut, pustakawan hanya mengecek tanpa menggeser.
    Buku rakUrut[] = {
        {101, "Laskar Pelangi"}, {102, "Bumi Manusia"}, {103, "Negeri 5
Menara"},
        {104, "Filosofi Teras"}, {105, "Hujan"}
    };

    cout << "=== BEST CASE (Data Terurut) ===" << endl;
    insertionSort(rakUrut, n);
    tampilkanRak(rakUrut, n);

    // SCENARIO 2: WORST CASE (O(n^2))
    // Data terbalik, setiap buku harus digeser sampai ke ujung kiri.
    Buku rakTerbalik[] = {
        {105, "Hujan"}, {104, "Filosofi Teras"}, {103, "Negeri 5 Menara"},
        {102, "Bumi Manusia"}, {101, "Laskar Pelangi"}
    };

    cout << "=== WORST CASE (Data Terbalik) ===" << endl;
    insertionSort(rakTerbalik, n);
    tampilkanRak(rakTerbalik, n);

    return 0;
}
```



STUDI KASUS

Mas Rehan seorang Ceo, ia memiliki perusahaan yang bergerak di bidang rumah tangga, saat ini ia sedang kebingungan untuk mengatur stok bahan di gudangnya saat ini Mas Rehan memiliki stok (dalam ton) :

- Beras (45)
- Jagung (60)
- Kentang (15)
- Singkong(20)

Sekarang tolong bantu mas rehan untuk membuat:

1. Buatlah ilustrasi sorting terbaik untuk mengatur stok dari yang paling sedikit
2. Program sorting terbaik agar mampu memudahkan dia untuk mengatur stok dari yang paling sedikit